

The architecture has two testable boundaries. Runtime tests verify that an invocation chooses the right operation with the right data. Transport tests verify that this operation becomes a valid Slack HTTP or WebSocket exchange. Persistence tests reopen the actual database. None of these should require posting administrative changes to the real workspace.

3. Code reading map

Start with `pkg/slackbot/model.go`. It defines the invocation kinds, message payloads, modal views, service interfaces and stable errors. Next read `internal/jsslack/module.go` for registration and `internal/jsslack/dispatch.go` for invocation routing, asynchronous host operations, state and reply ownership.

Read `internal/slacktransport/run.go` after understanding those types. It decodes Slack envelopes and handles ACKs. `client.go` implements actual Slack calls. `pkg/slackbot/recording.go` implements the same supported services as inspectable offline effects. New services need corresponding live and test implementations rather than special branches inside each bot.

The CLI lives in `pkg/slackcli`. `discover.go` inspects all candidate scripts, `commands.go` generates manifests and executes operations, `run_remote.go` resolves the selected profile, and `update_manifest.go` synchronizes app configuration before connecting. `internal/slackconfig/store.go` separates credential values from profile/app/installation metadata.

Finally, read the source bot itself, including its lib modules. The inventory script `scripts/01-inventory.py` records 156 syntactically recognizable registrations in `sources/discord-handlers.json`. This is an index, not a proof of completeness: computed IDs, imported helpers and dynamically registered handlers require source inspection.

4. Feature matrix by bot

Announcements, ping and hater

The announcements example produces a preview from a root-level script. Preserve that discovery pattern and the title/content fields using Block Kit. Do not turn a preview into a channel broadcast. Tests should verify that its only operation is the intended reply.

Ping is an API demonstration, not only a pong handler. Its source includes echo, feedback forms, search/autocomplete, announcements and component examples. Slack's corresponding controls must be demonstrated through commands, buttons, modals, explicit posting and supported selection/option callbacks. An external options callback is different from a Discord autocomplete interaction and requires its own decoder and response payload.

Hater contains contempt reports, self/target roasts, reluctant compliments, buttons, apology fields and message triggers. The initial Slack port is missing target-user selection and complete apology response behavior. Preserve the targeting choice using native user controls or parsed Slack user references. For passive triggers, subscribe only to the message event kinds actually needed and ignore the bot's own messages. Validation must not accept an empty apology merely because the modal can close.

Interaction types and unified demo

The interaction-types example exercises structured slash arguments, subcommands, a user context operation and a message context operation. Slack slash commands receive text; define and document a parser for `/fun roll 6` instead of pretending typed Discord arguments exist. A Slack message shortcut can implement quoting the selected message. Slack does not